

UNIVERSITATEA DIN ORADEA
FACULTATEA DE INGINERIE ELECTRICĂ ȘI TEHNOLOGIA INFORMAȚIEI
PROGRAMUL DE STUDIU: TEHNOLOGIA INFORMAȚIEI
FORMA DE ÎNVĂȚĂMÂNT: ÎNVĂȚĂMÂNT CU FRECVENȚĂ

Proiect de diplomă

Coordonator științific:
Prof.dr.habil. Popescu Daniela-Elena

Absolvent:
Popescu David-Sebastian

ORADEA 2024

UNIVERSITATEA DIN ORADEA
FACULTATEA DE INGINERIE ELECTRICĂ ȘI TEHNOLOGIA INFORMAȚIEI
PROGRAMUL DE STUDIU: TEHNOLOGIA INFORMAȚIEI
FORMA DE ÎNVĂȚĂMÂNT: ÎNVĂȚĂMÂNT CU FRECVENȚĂ

Aplicație web pentru detectarea bolilor cardiovasculare

Coordonator științific:

Prof.dr.habil. Popescu Daniela-Elena

Absolvent:

Popescu David-Sebastian

ORADEA 2024

CUPRINS

Capitolul I. Introducere	2
Capitolul II. Tehnologii Utilizate.....	3
HTML5.....	3
CSS3.....	3
JavaScript	4
Bootstrap	4
Python.....	4
Django	5
SQLite	6
Capitolul III. Elemente de machine learning	7
Analiza datelor	7
Preprocesarea datelor	9
Eliminarea valorilor anormale	9
Normalizarea datelor	11
Implementarea modelelor	11
Linear Regression	12
Random Forest.....	14
XGBoost.....	16
Capitolul IV. Analiza și proiectarea aplicației	18
Scopul și obiectivele aplicației	18
Arhitectura aplicației	19
Structura aplicației.....	20
Capitolul V. Implementarea Aplicației	23
Modulul administrativ	24
Dashboard-ul administrativ	25
Paginile de administrare a utilizatorilor.....	25
Pagina de administrare a grupurilor.....	26
Pagina de administrare a pacienților	27
Modulul de autentificare	28
Pagina de înregistrare	29
Pagina de autentificare.....	30
Pagina de cerere resetare a parolei	31
Pagina de confirmare resetare a parolei.....	31
Pagina de administrare a profilului.....	32
Pagina de deautentificare	32
Modulul de Pacienți	33
Pagina principală	33
Pagina detalii pacient.....	36
Pagina “Despre noi”	40
Capitolul VI. Concluzii	41
Bibliografie	43

Capitolul I. Introducere

În această lucrare mi-am propus să realizez o platformă web destinată detectării bolilor cardiovasculare, care să genereze grafice cu datele de analize și riscul de boală care reiese din acestea, cu scopul de a facilita diagnosticarea și îmbunătăți comunicarea între medic și pacient.

Bolile cardiovasculare reprezintă una dintre principalele cauze de deces la nivel mondial, iar identificarea și acționarea timpurie asupra problemelor cardiovasculare sunt esențiale pentru prevenirea complicațiilor grave.

Sistemul de detectare a bolilor cardiovasculare este menit să funcționeze ca un instrument centralizat de colectare și analiză a datelor medicale. Beneficiile unui astfel de sistem includ accesul rapid la istoricul medical al pacientului, generarea automată a rapoartelor și graficelor detaliate și creșterea șansei unei diagnosticări timpurii.

Scenariul cu care aplicația a fost proiectată în minte a fost pentru centrele de analize medicale pentru cu scopul de a complementa analiza profesioniștilor medicali și facilita colectarea, stocarea, analiza și comunicarea acestora cu pacienții privind rezultatele analizelor și riscurile survenite acestora. Acest lucru asigură că fiecare raport generat este verificat de un specialist, evaluat pentru potențiale erori și prezentat alături de interpretarea pertinentă a persoanelor calificate.

În următoarele capitole elementele și funcționalitatea acestui proiect sunt prezentate în ordine, începând cu tehnologiile utilizate, continuând cu elementele de machine learning și aspectele tehnice și arhitecturale ale aplicației, urmate de prezentarea aplicației în sine și finalizând cu concluziile obținute din implementarea proiectului.

Capitolul II. Tehnologii Utilizate

HTML5

Pentru structurarea și organizarea conținutului paginilor web am utilizat limbajul structural HTML5[1].

Printre cele mai mari avantaje pe care îl oferă limbajul este compatibilitatea sa cu marea majoritate a browserelor, vechi sau noi, și a extensiilor acestora. Aceste lucruri în conjunctură cu elementele oferite pentru SEO(Search Engine Optimization), precum elementul <title> care comunică titlul paginii motorului de căutare, pot contribui la promovarea paginii.

Limbajul este de asemenea integrat nativ cu tehnologii precum JavaScript și CSS3, asupra cărora voi elabora mai jos, și permite un control granular asupra modului în care elementele din pagină sunt afișate ceea ce ajută în crearea unei interfețe plăcute vizual.

Limbajul este de asemenea integrat, datorită utilizării la scară largă și cu majoritatea framework-urilor folosite în crearea aplicațiilor web, printre care și framework-ul Django, folosit în cadrul acestui proiect, ale cărui avantaje vor fi descrise mai jos.

Nu în ultimul rând important de menționat în procesul de alegere al limbajului este că sintaxa este simplă și aerată datorită elementelor de secționare oferite, printre care <footer>, <header> și <nav>. Acest lucru face codul ușor de scris și de citit ceea ce ajută mentenabilitatea proiectelor ce folosesc limbajul și ușurează dezvoltarea unui produs de calitate.

CSS3

Stilizarea paginilor web a fost realizată utilizând CSS3[2], limbaj care permite aplicarea de stiluri avansate, efecte vizuale și animații asupra elementelor paginii cu scopul de a îmbunătăți aspectul paginii și interactivitatea interfeței utilizatorilor.

CSS3 este integrat nativ cu HTML5 și permite definirea stilurilor elementelor de pagină individual, pentru fiecare element în parte, prin folosirea atributului “style” atunci când este necesar controlul granular al aspectului unui element sau cu ajutorul claselor de elemente, definite în cadrul documentului cu tag-ul <style> sau extern și apoi referențiat în document, când se dorește acest lucru.

Acest lucru permite dezvoltarea ușoară a unui website plăcut vizual și control granular asupra stilurilor diverselor elemente din pagina creată.

JavaScript

Pentru a crea elemente dinamice în pagină, precum pop-up-uri și tab-uri, am ales să folosesc JavaScript[3], un limbaj de programare care poate executa cod în timp real pentru a manipula documentul și este suportat de majoritatea browserelor moderne.

Limbajul JavaScript este de asemenea integrat nativ cu HTML5, lucru care permite definirea funcțiilor de JavaScript direct în cadrul documentului în tag-ul `<script>`, sau extern și referențiat ulterior. Acest lucru ușurează atât scrierea de cod cât și structurarea proiectului ceea ce contribuie la mentenabilitate.

Bootstrap

La implementarea interfeței, am integrat în proiect și framework-ul opensource Bootstrap[4], care oferă clase de stil și funcții JavaScript pentru stilizarea paginilor.

Avantajul principal al framework-ului Bootstrap constă în modularitatea și ușurința de implementare a elementelor definite de acesta. Clasele CSS ,predefinite de Bootstrap, pot fi adăugate prin simpla apelare a atributului “class” care primește ca parametru numele clasei pe care dorim să o utilizăm. Elementele ce folosesc JavaScript necesita de cele mai multe ori pe lângă atributului “class” cu clasa corespunzătoare a elementului dorit și attribute care să specifice tipul de operație care trebuie realizat și elementele din document care să fie afectate de operația făcută, însă parametrii folosiți pot varia în funcție de operația dorită.

Un alt avantaj al framework-ului Bootstrap este ușurința cu care acesta poate fi integrat în cadrul proiectelor care îl folosesc, fiind necesară simpla adăugare a unui link de referință către pagina de CSS sau JavaScript definită de framework, care este disponibilă public.

Python

Pentru prelucrarea și transferul datelor din baza de date către utilizator am ales să folosesc Python[5].

Python este un limbaj de programare de nivel înalt, cunoscut pentru sintaxa sa clară și versatilitatea în diverse domenii de aplicație.

Am ales Python în principal pentru că este utilizat pe scară largă în domenii precum analiza datelor, inteligența artificială și machine learning și dispune astfel de resursele necesare pentru crearea facilă a unei aplicații web care utilizează aceste tehnologii.

Printre bibliotecile importante pe care le-am utilizat în implementarea acestui proiect sunt:

- pandas și numpy, pentru procesarea și analiza datelor:
 - pandas[6] oferă structuri de date utile, cum ar fi DataFrame și Series, care facilitează manipularea și analiza datelor.
 - numpy[7] este utilizat pentru operații matematice și statistice avansate, oferind suport pentru array-uri multidimensionale și funcții matematice complexe.
- Joblib, Scikit-learn, XGBoost, pentru crearea și salvarea modelelor de predicție:
 - Scikit-learn[8] oferă diverse modele de clasificare, regresie și clustering și funcții pentru măsurarea performanțelor algoritmilor de machine learning.
 - XGBoost[9] oferă un algoritm de boosting eficient și scalabil, este cunoscut pentru performanțele sale înalte în competițiile de machine learning.
 - Joblib[10] este utilizat pentru salvarea și încărcarea rapidă a modelelor generate cu algoritmi de mai sus reutilizarea modelelor de machine learning.
- Matplotlib[11], pentru vizualizarea datelor, și crearea de grafice statice pentru interpretarea și prezentarea datelor generate de aplicație și modele. Matplotlib oferă o gamă largă de tipuri de grafice, cum ar fi grafice de tip lineplot, barplot, scatterplot și histogram, care pot fi personalizate în funcție de nevoile utilizatorului.

Django

În implementarea aplicației am folosit, pentru infrastructura de bază, framework-ul Django[12], scris în Python.

Django este un framework web opensource de nivel înalt care implementează modelul arhitectural MVT(Model-View-Template) pentru administrarea proceselor de transfer și prelucrare a datelor aplicației, oferind un schelet infrastructural robust cu multe funcționalități predefinite cu ajutorul cărora putem construi aplicații complexe cu ușurință.

Django oferă aceste funcționalități printr-un set de clase și funcții predefinite care pot fi moștenite și modificate pentru a se adapta nevoilor specifice ale aplicației. Aceste clase implementează modele pentru formulare și tabele pentru interacțiunea cu baze de date, care definesc metode prin intermediul cărora datele aplicației pot fi procesate. Procesarea acestor date se poate face și în afara claselor cu ajutorul funcțiilor definite de utilizator sau de framework, lucru comun în particular în view-uri, înainte ca datele să fie trimise către template-uri care randează practic paginile vizualizate de utilizator.

În particular Django include suport pentru autentificare, autorizare și administrarea utilizatorilor, actualizate constant pentru a menține un nivel înalt de securitate și care pot fi augmentate ușor prin metodele descrise mai sus, facilitând astfel crearea rapidă a funcționalităților esențiale pentru aplicațiile web.

SQLite

Pentru stocarea și gestionarea datelor am ales să folosesc ca și sistem de administrare a bazelor de date relaționale SQLite[13].

SQLite este gratuit și open-source oferind o soluție eficientă pentru stocarea și administrarea datelor. SQLite este ideal pentru dezvoltarea și testarea aplicațiilor la scară redusă, datorită simplității sale și faptului că nu necesită un server separat pentru a funcționa. Baza de date SQLite este stocată într-un singur fișier pe disc, ceea ce facilitează gestionarea și distribuția aplicației.

Printre avantajele SQLite se numără și performanțele remarcabile pe care le oferă pentru aplicațiile cu volum mediu de date și un nivel mediu de interogări simultane.

Capitolul III. Elemente de machine learning

Pentru a maximiza performanța și precizia modelelor am utilizat în cadrul proiectului de diplomă diverse tehnici de preprocesare și transformare și modelare a datelor, cu scopul de a prezice riscul de boală cardiovasculară, procese care vor fi detaliate mai jos.

Analiza datelor

Înainte de a decide stagiile de preprocesare necesare pentru a putea implementa algoritmi de predicție este necesară analiza atentă a datelor, în particular importante fiind tipurile de date utilizate, intervalele între care fiecare categorie are valorile și distribuția datelor pe aceste intervale.

Primul pas în analiza datelor este verificarea setului de date pentru valori lipsă, întrucât prezența acestora poate conduce la erori în interpretarea datelor și în modelul de predicție. În cazul setului de date folosit nu există valori lipsă deci se poate trece la următorul pas.

Avem în tabelul de mai jos primele 5 rânduri ale setului de date:

Vârstă	Sex	Înălțime	Greutate	TAS	TAD	Colesterol	Glucoză	Fumător	Băutor	Activ	Risc_Boală
18393	2	168	62	110	80	1	1	0	0	1	0
20228	1	156	85	140	90	3	1	0	0	1	1
18857	1	165	64	130	70	3	1	0	0	0	1
17623	2	169	82	150	100	1	1	0	0	1	1
17474	1	156	56	100	60	1	1	0	0	0	0

Câmpurile de mai sus reprezintă datele pacientului astfel: vârsta este trecută în zile, sexul pacientului este 1 dacă este bărbat și 2 dacă este femeie, înălțimea este măsurată în centimetri, greutatea este măsurată în kilograme, tensiunea arterială sistolică (TAS) și distolică (TAD) sunt măsurate și trecute fără a fi transformate, colesterolul și glucoza sunt trecute pe categorii, cu 1 reprezentând valori normale, 2 valori peste limita normală și 3 valori mult peste limita normală, coloanele fumător, băutor și activitate sunt valori binare pentru prezența acestor caracteristici în stilul de viață al pacientului, iar coloana Risc_Boală este valoarea prezisă înainte de conversia în date de regresie și clasifică pacienții în funcție de prezența binară a bolii.

Observăm ca datele sunt împărțite în funcție de tip între date binare, date pe categorii și date liniare. În particular vizibilă este diferența vastă de intervale asumată de datele liniare care ar putea afecta drastic valoarea anumitor caracteristici cu intervale mai largi în interpretarea lor de către model, în special în cazul algoritmilor care folosesc arbori de decizie pentru aceste caracteristici.

Un alt factor important în decizia algoritmilor de preprocesare folosiți este distribuția datelor pe valori, lucru care poate fi vizualizat în particular ușor cu ajutorul unui grafic de tip scatterplot, precum cel vizibil în figura 3.1.

În acest caz am decis să folosim un grafic care evaluează și densitatea datelor pe zone și le reprezintă prin adăugarea unei linii care reprezintă numărul de valori în zonele în care datele sunt distribuite.

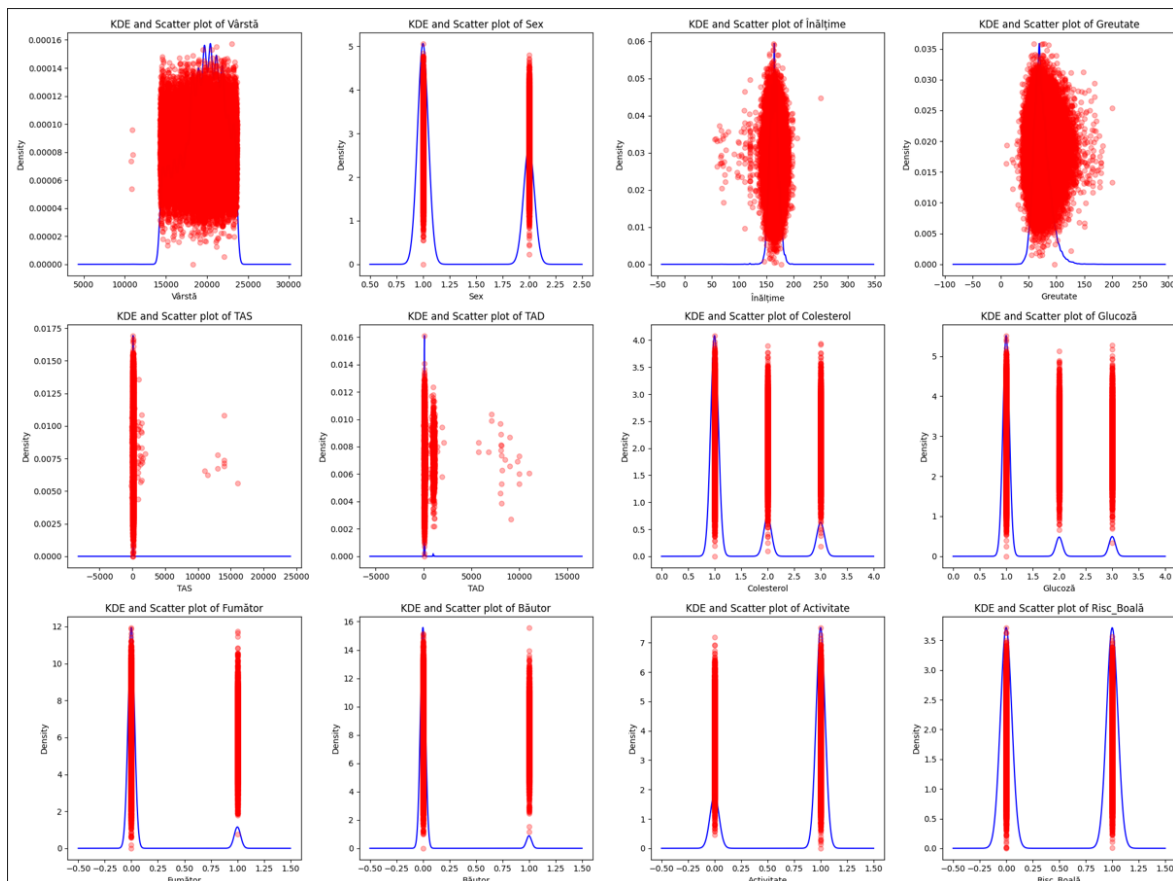


Figura 3.1

Analiza graficului conduce la concluzia că datele sunt distribuite normal, cu puține valori anormale, însă observăm și aici diferențele de scală între intervalele folosite de caracteristicile cu date liniare ale pacienților, deci putem concluziona că setul de date este calitativ și nu necesită prea multă preprocesare, dar că ar fi util un algoritm de normalizare a valorilor pentru caracteristicile cu valori continue.

Preprocesarea datelor

Pentru utilizarea datelor în crearea unui algoritm de predicție a bolilor date am concluzionat în urma analizei că este necesară aplicarea unor algoritmi de normalizare a datelor.

Luând în considerare faptul că algoritmi de normalizare sunt afectați disproporționat de datele cu valori extreme ale setului de date am decis că un algoritm de eliminare a valorilor anormale, puține dar observabile, este necesar pentru a evita efectul disproporționat pe care aceste valori l-ar avea asupra setului de date normalizat.

Eliminarea valorilor anormale

Pentru eliminarea valorilor anormale din setul de date am decis să folosesc Isolation Forest[14].

Algoritmul generează arbori de decizie în așa fel încât fiecare intrare a setului de date să ajungă izolată pe o frunză a arborelui. Pentru a asigura că nu favorizează anumite caracteristici ale setului de date în selecția acestor valori anormale algoritmul construiește mulți arbori și decide, la fiecare pas, caracteristicile cu ajutorul cărora se generează nodul în mod aleator. După generarea numărului de arbori dorit de utilizator algoritmul asumă că intrările care ajung consistent să se afle pe frunze sunt anormale deoarece au valori extreme și elimină valori dintre aceste frunze până când ajunge la procentajul specificat de utilizator.

Am ales acest algoritm în principal datorită eficienței și a abilității de a detecta valori anormale compuse de multiple caracteristici, astfel că elimină valorile extreme indiferent dacă doar un parametru le diferențiază sau dacă există mai mulți factori, până când datele pot fi considerate normale conform procentajului stabilit la apelare.

Implementarea programului a fost realizată folosind funcția `isolation_forest`[15] din pachetul `sklearn.ensemble`, parametrizată cu asumția că procentajul de contaminare este 0.1% estimare derivată din graficul realizat la analiză, iar pentru replicabilitate am specificat valoarea parametrului `random_state` să fie 42 ca algoritmul să amestece datele în aceeași manieră înainte de generarea arborilor când este apelat, după cum se poate observa și în figura 3.2.

```

import pandas as pd
from sklearn.ensemble import IsolationForest

data = pd.read_csv('Models/Datasets/processed_cardio_train.csv')

iso_forest = IsolationForest(contamination=0.001, random_state=42)
iso_forest.fit(data)

outliers = iso_forest.predict(data)

inliers_mask = outliers == 1

inliers_data = data[inliers_mask]

print("Number of inliers:", len(inliers_data))
print(inliers_data.head())

inliers_data.to_csv('Models/Datasets/processed_cardio_train.csv', index=False)

```

Figura 3.2

```

import pandas as pd
from sklearn.preprocessing import StandardScaler
import joblib

continuous_columns = ['Vârsta', 'Înălțime', 'Greutate', 'TAS', 'TAD']
categorical_columns = ['Colesterol', 'Glucoză']
binary_columns = ['Sex', 'Fumător', 'Băutor', 'Activitate']

def normalize_columns(df, scaler=None):
    if scaler is None:
        scaler = StandardScaler()
        df[continuous_columns] = scaler.fit_transform(df[continuous_columns])
        return df, scaler
    else:
        df[continuous_columns] = scaler.transform(df[continuous_columns])
        return df

csv_path = 'Models/Datasets/processed_cardio_train.csv'
output_path = 'Models/Datasets/normalized_cardio_train.csv'
scaler_path = 'Models/Models/scaler.pkl'

df = pd.read_csv(csv_path)

features = df.drop(columns=['Risc_Boală'])
target = df['Risc_Boală']

normalized_features, scaler = normalize_columns(features)
joblib.dump(scaler, scaler_path)

normalized_df = pd.concat([normalized_features, target], axis=1)

normalized_df.to_csv(output_path, index=False)

```

Figura 3.3

Normalizarea datelor

Pentru stagiul de normalizare al valorilor continue din setul de date am decis, după detectarea și eliminarea valorilor anormale, să folosim algoritmul StandardScaler[16].

Standard Scaler normalizează datele prin scăderea mediei valorilor la 0 și împărțirea valorilor rezultate cu deviația acestora, pentru a obține date scalate, cu medie de 0 și deviație de 1.

Avantajul principal al acestui algoritm este eficiența și faptul că asigură că contribuția datelor la model nu este alterată atât timp cât datele nu conțin o cantitate disproporționată de valori anormale.

Implementarea programului de normalizare a fost realizată cu ajutorul funcției StandardScaler[17] din pachetul sklearn.preprocessing și poate fi observată în figura 3.3.

Implementarea modelelor

Pentru implementarea algoritmului de predicție am implementat trei modele cu diverse grade de complexitate cu scopul de a compara atât acuratețea rezultatelor prezise cât și performanțele acestora.

Deoarece dorim să prezicem riscul de boală al pacientului, iar datele măsoară prezența bolii binar este necesar ca, în timpul pasului de modelare, înainte de generarea modelului, să convertim datele pentru a putea fi utilizate de modelele de regresie implementate. Această conversie trebuie făcută pentru fiecare model în parte folosind algoritmi de predicție a probabilității compatibili, pentru a nu introduce eroare în algoritmii de predicție.

Pentru evaluarea performanței modelelor folosim următoarele metrici[18]:

- Mean Absolute Error (MAE): Măsoară diferența medie absolută între valorile prezise și cele reale.
- Mean Squared Error (MSE): Măsoară diferența medie pătratică între valorile prezise și cele reale, penalizând erorile mari mai sever decât MAE
- Root Mean Squared Error (RMSE): Este rădăcina pătrată a MSE, oferind o interpretare a erorii în aceleași unități ca și variabila prezisă, ceea ce îl face mai ușor de înțeles.
- Normalized Root Mean Squared Error (NRMSE): Este RMSE cu intervalul de valori al datelor și valoarea medie a acestora normalizate, ceea ce permite compararea performanței între modele diferite.

Pentru asigurarea consistenței în măsurarea performanței modelelor utilizăm o tehnică numită KFold Cross-Validation[19] care împarte setul de date în K secțiuni, fiecare secțiune fiind împărțită la rândul ei în date de antrenament și test, repetând procesul de antrenament și test de K ori și returnând valorile medii calculate pe durata acestui proces.

Linear Regression

Primul model pe care l-am utilizat în predicția riscului de boală cardiovasculară este un model de Linear Regression[20].

Algoritmul ia fiecare caracteristică în parte și compară valorile acestei caracteristici cu valoarea prezisă, apoi potrivește o linie care estimează corelarea dintre valoarea prezisă și caracteristica selectată. Linia estimată este cea care are suma distanțelor față de toate punctele minimă.

Avantajul principal al acestui model constă în simplitatea sa și ușurința cu care parametrii și rezultatele acestuia pot fi interpretate, astfel că poate ajuta în stabilirea unui punct solid de referință și în estimarea liniarității și a corelării directe între valoarea prezisă și caracteristicile modelului.

Pentru conversia datelor de clasificare inițiale în datele de regresie necesare pentru a prezice riscul de boală cardiovasculară al pacientului am decis să folosesc funcția de prezicere a probabilității atribuită algoritmului de Logistic Regression[21], care este un algoritm liniar de clasificare. Această abordare funcționează bine deoarece Logistic Regression și Linear Regression funcționează prin corelarea directă a caracteristicilor modelului cu valoarea prezisă și folosesc funcții liniare pentru a estima valoarea acestei corelații, cu diferența principală fiind că algoritmul de Logistic Regression aplică o sigmoidă asupra acestei funcții pentru a rotunji rezultatul într-unul binar.

Implementarea programului de conversie a datelor a fost realizată cu ajutorul funcției LogisticRegression[22] din pachetul `sklearn.linear_model` și poate fi observată în figura 3.4, iar algoritmul de prezicere a fost realizat cu ajutorul funcției LinearRegression[23] din pachetul `sklearn.linear_model` și poate fi observată în figura 3.5.

Procesul de evaluare al acestui model returnează următoarele valori:

- Mean Absolute Error (MAE): 0.09380069972598332
- Mean Squared Error (MSE): 0.015437431027882623
- Root Mean Squared Error (RMSE): 0.12424079306588327
- Normalized RMSE (NRMSE): 0.24928088466354464

Aceste valori dezvăluie că precizia modelului este imperfectă, cauza principală fiind cel mai probabil limitările modelului în interacțiunile cu datele de clasificare, ale căror distribuție este neliniară.

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

data = pd.read_csv('Models/Datasets/normalized_cardio_train.csv')

X = data.drop('Risc_Boală', axis=1)
y = data['Risc_Boală']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

logistic_model = LogisticRegression(max_iter=1000)
logistic_model.fit(X_train, y_train)

full_probabilities = logistic_model.predict_proba(X)[: , 1]

data['Risc_Boală'] = full_probabilities

data.to_csv('Models/Datasets/logistic_processed_cardio_train.csv', index=False)

print(full_probabilities)

```

Figura 3.4

```

import pandas as pd
from sklearn.linear_model import LinearRegression
import joblib

data = pd.read_csv("Models/Datasets/logistic_processed_cardio_train.csv")

X = data.drop(columns=['Risc_Boală'])
y = data['Risc_Boală']

model = LinearRegression()
model.fit(X, y)

joblib.dump(model, 'Models/Models/Linear_Regression/linear_model.pkl')

```

Figura 3.5

Random Forest

Al doilea model pe care l-am utilizat în predicția riscului de boală cardiovasculară este RandomForestRegression[24].

RandomForestRegression este un algoritm care utilizează arbori de decizie generați aleatoriu pentru a face predicții precise și stabile. Procesul începe cu un stadiu de bootstrapping, proces care eșantionează aleator din datele de antrenament permițând duplicate, dar menținând numărul total de intrări folosit la antrenament. Pentru fiecare eșantion, se construiește un arbore de decizie folosind o submulțime aleatoare de caracteristici, ceea ce minimizează suprapunerea între arbori, care sunt utilizați pentru a calcula valoarea prezisă.

Valoarea acestui algoritm constă în faptul că combină avantajele arborilor de decizie care oferă avantaje mari în interpretarea caracteristicilor cu date clasificare, minimizând pierderea de performanță pentru caracteristicile cu date liniare, prin utilizarea arborilor independenți care contribuie la algoritm în funcție de rezultatul arborelui de decizie pentru coloanele folosite ca și input.

Pentru conversia datelor de clasificare în date de regresie pentru predicția riscul de boală cardiovasculară al pacientului am decis să folosesc funcția de predicție a probabilității oferită de algoritmul RandomForestClassification pentru a asigura compatibilitatea metodelor cu care datele sunt convertite și prelucrate.

Implementarea programului de conversie a datelor a fost realizată cu ajutorul funcției RandomForestClassifier[25] din pachetul sklearn.ensemble și poate fi observată în figura 3.6, iar algoritmul de prezicere a fost realizat cu ajutorul funcției RandomForestRegressor[26] din pachetul sklearn.ensemble și poate fi observată în figura 3.7.

Procesul de evaluare al acestui model returnează următoarele valori:

- Mean Absolute Error (MAE): 0.1870578242352103
- Mean Squared Error (MSE): 0.05610221903958136
- Root Mean Squared Error (RMSE): 0.23685096466099037
- Normalized RMSE (NRMSE): 0.4761162824077607

Aceste valori dezvăluie că performanța modelului este departe de ideală, motivul principal fiind cel mai probabil interconectivitatea nativă a datelor medicale, algoritmul având, în general performanțe slabe, în gestionarea relațiilor pe acest tip de date.

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

data = pd.read_csv('Models/Datasets/normalized_cardio_train.csv')

X = data.drop('Risc_Boală', axis=1)
y = data['Risc_Boală']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

rf_classifier = RandomForestClassifier(n_estimators=250, random_state=42)
rf_classifier.fit(X_train, y_train)

full_probabilities = rf_classifier.predict_proba(X)[: , 1]

data['Risc_Boală'] = full_probabilities

data.to_csv('Models/Datasets/rf_processed_cardio_train.csv', index=False)

print(full_probabilities)

```

Figura 3.6

```

import pandas as pd
from sklearn.ensemble import RandomForestRegressor
import joblib

data = pd.read_csv("Models/Datasets/rf_processed_cardio_train.csv")

X = data.drop(columns=['Risc_Boală'])
y = data['Risc_Boală']

model = RandomForestRegressor(n_estimators=250, random_state=42)
model.fit(X, y)

joblib.dump(model, 'Models/Models/Random_Forest/rf_model.pkl')

```

Figura 3.7

XGBoost

Cel de-al treilea model pe care l-am utilizat în predicția riscului de boală cardiovasculară este XGBoost[27] pentru regresie.

Algoritmul începe prin antrenarea unui arbore de decizie simplu care face predicții de bază. Ulterior, algoritmul generează și adaugă succesiv arbori de decizie, fiecare încercând să corecteze erorile făcute de arborii anteriori, proces cunoscut sub numele de gradient boosting. Fiecare nou arbore este antrenat pe erorile arborilor anteriori, îmbunătățind treptat acuratețea modelului.

Valoarea acestui algoritm constă în faptul că combină predicțiile mai multor arbori pentru a obține un model final robust și precis și în capacitatea sa de a interpreta relațiile dintre date corect prin autocorectare.

Un alt avantaj al XGBoost este faptul că implementează nativ elemente de paralelizare lucru care oferă avantaje mari în eficiență și timp de execuție.

Pentru conversia datelor de clasificare în date de regresie necesară pentru predicția riscului de boală cardiovasculară al pacientului, am utilizat funcția de predicție a probabilității oferită de algoritmul XGBoost pentru clasificare lucru care asigură compatibilitatea metodelor de conversie și prelucrare a datelor.

Implementarea programului de conversie a datelor a fost realizată cu ajutorul funcției XGBoost pentru clasificare din biblioteca xgboost[28] și poate fi observată în figura 3.8, iar algoritmul de prezicere a fost realizat cu ajutorul funcției XGBoost pentru regresie din aceeași bibliotecă poate fi observat în figura 3.9.

Procesul de evaluare al acestui model returnează următoarele valori:

- Mean Absolute Error (MAE): 0.04890584847044783
- Mean Squared Error (MSE): 0.00512670530718565
- Root Mean Squared Error (RMSE): 0.07153398202548752
- Normalized RMSE (NRMSE): 0.14358312027419057

Aceste valori dezvăluie că performanța modelului este de departe superioară algoritmilor utilizați anterior, probabil datorită capacităților modelului de a interpreta relații complexe între date, indiferent de tipul acestora.

```

import pandas as pd
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier

data = pd.read_csv('Models/Datasets/normalized_cardio_train.csv')

X = data.drop('Risc_Boală', axis=1)
y = data['Risc_Boală']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = XGBClassifier(use_label_encoder=False, eval_metric='logloss')

model.fit(X_train, y_train)

probabilities = model.predict_proba(X_test)

full_probabilities = model.predict_proba(X)[: , 1]

data['Risc_Boală'] = full_probabilities

print(full_probabilities)

data.to_csv('Models/Datasets/xgb_processed_cardio_train.csv', index=False)

```

Figura 3.8

```

import pandas as pd
import xgboost as xgb
import joblib

data = pd.read_csv("Models/Datasets/xgb_processed_cardio_train.csv")

X = data.drop(columns=['Risc_Boală'])
y = data['Risc_Boală']

model = xgb.XGBRegressor()
model.fit(X, y)

joblib.dump(model, 'Models/Models/XG_Boost/xgb_model.pkl')

```

Figura 3.9

Capitolul IV. Analiza și proiectarea aplicației

Scopul și obiectivele aplicației

Aplicația "CVD_Detector" este destinată utilizării în centrele de analize medicale, unde poate centraliza datele pacienților, ușura gestionarea datelor acestora și contribui la detecția și prevenția afecțiunilor cardiovasculare prin optimizarea procesului de diagnosticare și generarea de vizualizări utile pentru acest gen de date. Aplicația este menită să fie utilizată alături de analiza specialiștilor și a ușura comunicarea dintre aceștia și pacienții lor oferind date ușor de interpretat și ridicând un semnal de alarmă pentru pacienții cu predispoziție la acest tip de afecțiune.

Obiectivele aplicației includ:

- Procesarea automată și analiza datelor medicale pentru identificarea timpurie a semnelor de boală cardiovasculară.
- Centralizarea datelor pacienților, facilitând accesul la datele istorice ale pacientului pentru a facilita predicțiile medicilor.
- Generarea de rapoarte detaliate care pot fi interpretate ușor atât de medici cât și de pacienți.
- Asigurarea confidențialității și securității datelor pacienților.
- Oferirea unei interfețe intuitive și ușor de utilizat.

Funcționalități principale:

- Formulare validate, pentru introducerea și ștergerea datelor pacienților, alături de filtre parametrizate pentru a ușura căutările.
- Generarea de predicții precise a riscului de boală și a explicațiilor elementelor ce contribuie la acest risc.
- Generarea de rapoarte și grafice interactive care să fie ușor de interpretat.
- Gestionarea securizată a autentificării și autorizării utilizatorilor

Arhitectura aplicației

Aplicația utilizează arhitectura **Model-View-Template (MVT)**[29], un model de arhitectură specific Django, care separă aplicația în trei componente principale: Model, View și Template. Fiecare componentă este responsabilă pentru interacțiunea cu un nivel specific al aplicației, astfel :

- **Modelele** sunt utilizate pentru a defini și interacționa cu structura bazei de date SQLite care stochează toate informațiile necesare aplicației, inclusiv datele pacienților, informațiile utilizatorilor și log-urile de activitate. Această interacțiune este realizată prin intermediul ORM-ului (Object Relational Mapper) Django, care asigură conversia intrărilor tabelor din baza de date în obiecte ce pot fi utilizate de backend-ul aplicației și viceversa pentru a salva datele noi în baza de date.
- **View-urile** sunt responsabile pentru procesarea logicii aplicației și gestionarea fluxului de date între model și template. Django utilizează view-urile pentru a primi cereri de la utilizatori, a interacționa cu modelele și a returna datele necesare pentru ca template-urile să poată afișa corect datele cerute de utilizatori.
- **Template-urile** procesează datele primite de la view-uri și gestionează modul în care acestea sunt afișate utilizatorilor. Structura și stilizarea în cadrul acestora sunt realizate cu HTML5, CSS3 și JavaScript, iar datele returnate de view-uri în context pot fi accesate folosind declarații specifice framework-ului django.

Interacțiunea dintre aceste nivele poate fi vizualizată mai bine în figura 4.1, iar structura bazei de date, cu tabelele relaționale necesare pentru funcționarea aplicației poate fi observată în figura 4.2.

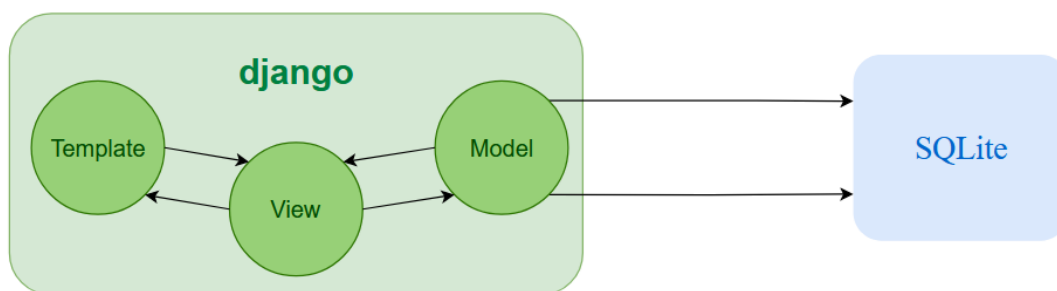


Figura 4.1

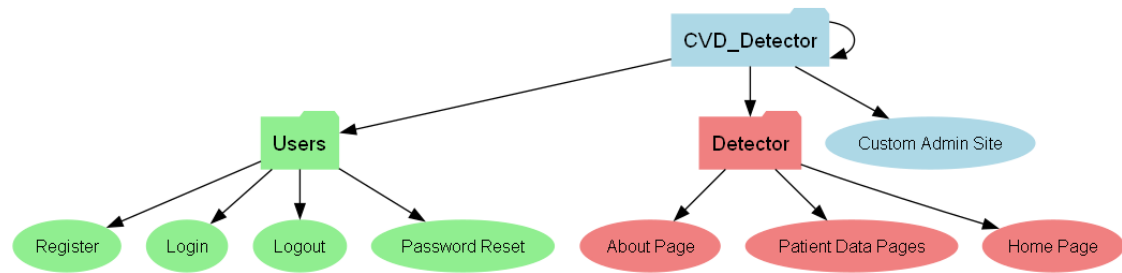


Figura 4.3

Modulul administrativ

Pentru gestionarea sarcinilor de administrare ce țin de aplicația "CVD_Detector" am creat modulul administrativ în rădăcina proiectului.

Pentru a ușura sarcinile administratorilor și crea un punct de acces unitar unde administratorii pot gestiona datele sistemului și evalua starea acestuia, am creat un dashboard vizibil doar utilizatorilor desemnați de aceștia.

Funcționalități cheie ale acestei componente sunt:

- Gestionarea datelor pacienților
- Adăugarea, modificarea și ștergerea utilizatorilor din sistem
- Definirea și atribuirea de permisiuni specifice diferitelor grupuri de utilizatori sau utilizatorilor specifici

Modulul de autentificare

Gestiunea sistemului de autentificare se realizează în cadrul modulului de autentificare.

Pentru a permite utilizatorilor platformei să efectueze activitățile necesare, care necesită acces date cu caracter sensibil, fără a compromite securitatea aplicației, este necesar să restricționăm accesul la componentele aplicației prin intermediul conturilor de utilizatori, care să conțină elementele și funcționalitățile necesare pentru o interacțiune securizată și fluidă a acestora cu aplicația.

Funcționalități cheie ale acestei componente sunt:

- Crearea unui cont inactiv, a cărui acces la datele aplicației și funcționare trebuie aprobată de administrator, pentru a nu compromite securitatea aplicației
- Actualizarea simplă și rapidă a datelor utilizatorului în cadrul sesiunii
- Resetarea parolei fără intervenția administratorului printr-un link trimis pe email-ul de contact, care folosește token-uri pentru a se asigura că nu este activ decât pe durată scurtă de la generare și că poate fi accesat o singură dată
- Autentificarea folosind token-uri de sesiune pentru a menține securitatea aplicației

Modulul de pacienți

Pentru gestiunea datelor pacienților și generarea de predicții și rapoarte am creat modulul de pacienți.

Pentru centralizarea datelor pacienților și integrarea unitară acestora cu componentele necesare pentru realizarea predicțiilor și graficelor am implementat modulul de pacienți.

Funcționalități cheie ale acestei componente sunt:

- Centralizează datele tuturor pacienților într-o interfață ce permite adăugarea ușoară a noilor pacienți și filtrarea datelor pentru a găsi ușor categoriile de pacienți dorite.
- Analizează datele pacienților pentru a evalua riscul de boli cardiovasculare.
- Generează rapoarte și grafice detaliate, vizualizate unitar, pentru fiecare pacient, cu scopul de a oferi pacienților și medicilor o imagine clară, care nu pierde contextul istoric al pacientului și este ușor de interpretat.

Capitolul V. Implementarea Aplicației

Datorită arhitecturii aplicației fiecare pagină a website-ului, randată de template-ul folosit pentru aceasta are un view unic care să genereze contextul necesar pentru aplicație, astfel consider că este de valoare ca logica pentru generarea elementelor interfeței și procesele de generarea a datelor să fie prezentate împreună.

Pentru a evita repetarea de cod ne folosim de un alt feature Django, care permite referențierea elementelor de cod și moștenirea structurilor de date de la alte template-uri. Elementele pe care le-am modificat, astfel, pentru a evita rescrierea lor sunt, bara de navigare care este definită în template-ul navbar.html, structura de bază a website-ului și referințele către bibliotecile și fișierele utilizate sunt definite template-ul base.html, care referențiază bara de navigare, iar pentru modificarea interfeței paginilor de administrare am suprascris template-ul de bază base.css, folosit de django nativ pentru a defini structura componentei de administrare, lucru care ne permite să referențiem și aici bara de navigare generată de noi și a genera o interfață unitară cu restul website-ului.

Putem vizualiza ușor modul în care aceste pagini moștenesc și referențiază elemente externe pentru a evita repetiția codului, în figura 5.1.

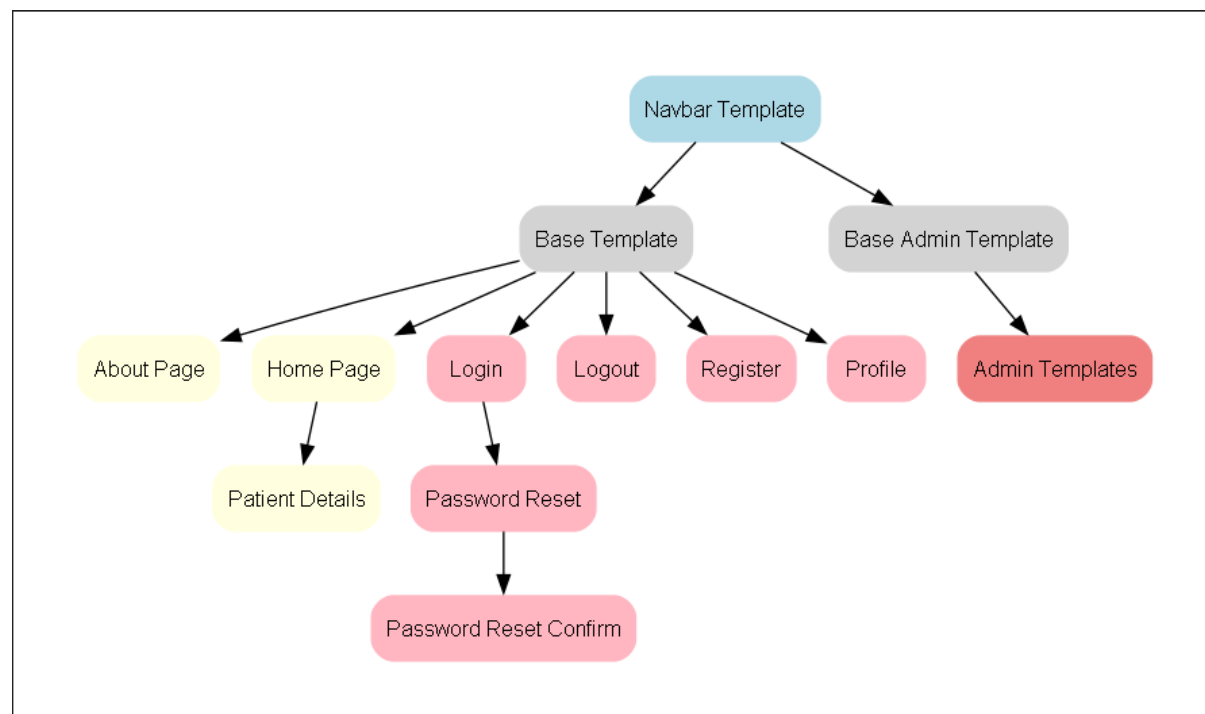


Figura 5.1

Modulul administrativ

Modulul administrativ predefinit Django, este oferit ca un pachet unitar, componentă a proiectelor Django care utilizează propriile pagini, cu modele și view-uri, pentru gestionarea accesului și propriul stil unitar, paginile fiind generate prin intermediul obiect site pentru fiecare model definit în cadrul proiectului. Pentru a integra componenta ca parte a website-ului, fără a pierde funcționalitățile valoroase oferite de aceste pagini, este necesară, nu doar modificarea template-ului de bază, ci implementarea unei clase personalizabile de administrare, care să implementeze paginile definite de noi și să elimine componentele nefolositoare ale interfeței.

Implementarea acestei clase este realizată prin crearea unei clase moștenitoare care preia elementele de la clasa de bază și suprascrie funcția care creează paginile modulului, pentru a referenția paginile dorite de noi, după cum se poate observa în figura 5.2.

```
class MyAdminSite(admin.AdminSite):
    index_title='Cardiotest'
    site_title='Administration'
    site_header='Cardiotest'
    def get_urls(self):
        def wrap(view, cacheable=False):
            def wrapper(*args, **kwargs):
                return super(MyAdminSite,self).admin_view(view, cacheable)(*args, **kwargs)
            wrapper.admin_site = self
            return update_wrapper(wrapper, view)

        urlpatterns = [
            path("", wrap(self.index), name="index"),
            path('login/', auth_views.LoginView.as_view(template_name='Users/login.html', form_class=UserLoginForm), name='login'),
            path('logout/', auth_views.LogoutView.as_view(template_name='Users/logout.html'), name='logout'),
            path('password-reset/', auth_views.PasswordResetView.as_view(template_name='Users/password_reset/password_reset.html', form_class=UserPasswordResetForm), name='password_reset'),
            path('password-reset/done/', auth_views.PasswordResetDoneView.as_view(template_name='Users/password_reset/password_reset_done.html'), name='password_reset_done'),
            path('password-reset-confirm///', auth_views.PasswordResetConfirmView.as_view(template_name='Users/password_reset/password_reset_confirm.html', form_class=UserSetPasswordForm), name='password_reset_confirm'),
            path('password-reset-complete/', auth_views.PasswordResetCompleteView.as_view(template_name='Users/password_reset/password_reset_complete.html'), name='password_reset_complete'),
            path("jsi18n/", wrap(self.i18n_javascript, cacheable=True), name="jsi18n"),
        ]

        valid_app_labels = []
        for model, model_admin in self._registry.items():
            urlpatterns += [
                path(
                    "%s/%s/" % (model._meta.app_label, model._meta.model_name),
                    include(model_admin.urls),
                ),
            ]
            if model._meta.app_label not in valid_app_labels:
                valid_app_labels.append(model._meta.app_label)

        if valid_app_labels:
            regex = r"^(?P<app_label>"+ "|".join(valid_app_labels) + ")/?$"
            urlpatterns += [
                re_path(regex, wrap(self.app_index), name="app_list"),
            ]

        if self.final_catch_all_view:
            urlpatterns.append(re_path(r"^(?P<url>.*)$", wrap(self.catch_all_view)))

        return urlpatterns
```

Figura 5.2

Pentru îmbunătăți securitatea modulului am adăugat și o componentă tip middleware, care interceptează request-urile de la utilizator și returnează o eroare 404 când se încearcă accesarea componentei dacă utilizatorul nu este logat sau contul nu are permisiuni de vizualizare a paginii, implementarea căreia poate fi observată în figura 5.3.

```
class AdminAccessMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if request.path.startswith('/admin/') and (not request.user.is_authenticated or not request.user.is_staff):
            raise Http404("Page not found")
        return self.get_response(request)
```

Figura 5.3

Implementarea acestor modificări permite integrarea unitară și securizată cu restul website-ului fără pierderea funcționalităților valoroase oferite de modul de administrare a diverselor date parte din aplicație, prin paginile specifice fiecărui model, care vor fi descrise în detaliu mai jos.

Dashboard-ul administrativ

Dashboard-ul administrativ, permite, după cum putem observa și în figura 5.4, vizualizarea și accesarea tuturor modelelor definite în cadrul proiectului precum și modulele din care acestea fac parte.



Figura 5.4

Paginile de administrare a utilizatorilor

Paginile de administrare a utilizatorilor, afișează lista de utilizatori din baza de date și permit administratorului să gestioneze ușor și organizat datele utilizatorilor, după cum se poate observa în figura 5.5. Administratorii pot adăuga utilizatori noi, șterge utilizatori existenți și modifica fiecare dintre acești utilizatori prin adăugarea în grupurile de utilizatori cu anumite permisiuni sau oferind permisiuni specifice utilizatorului, după cum se poate observa în figura 5.6, lucru realizat de Django prin alterarea câmpurilor din obiectul generat de ORM-ul Django care corespunde intrărilor din baza de date.

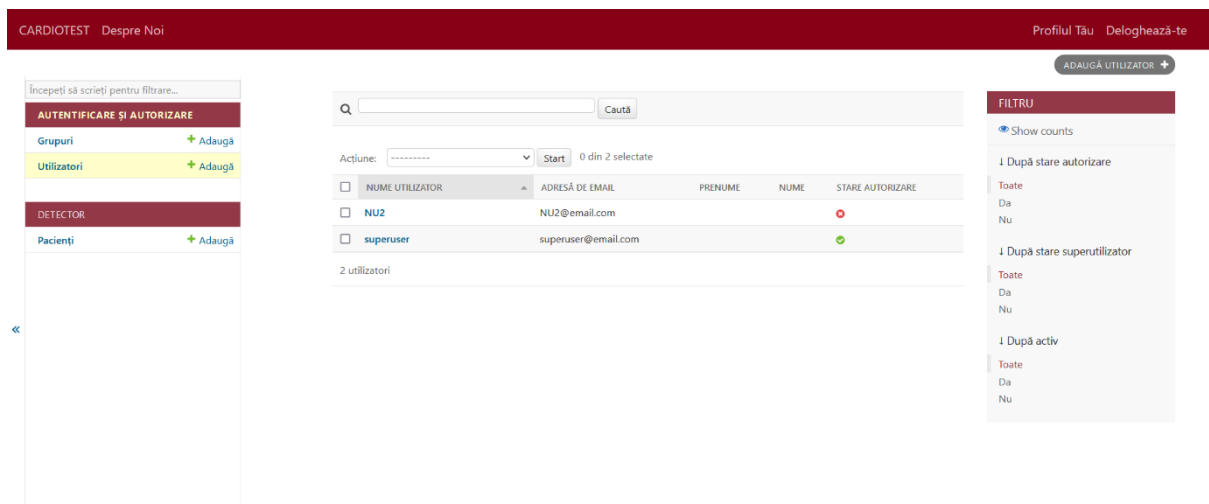


Figura 5.5

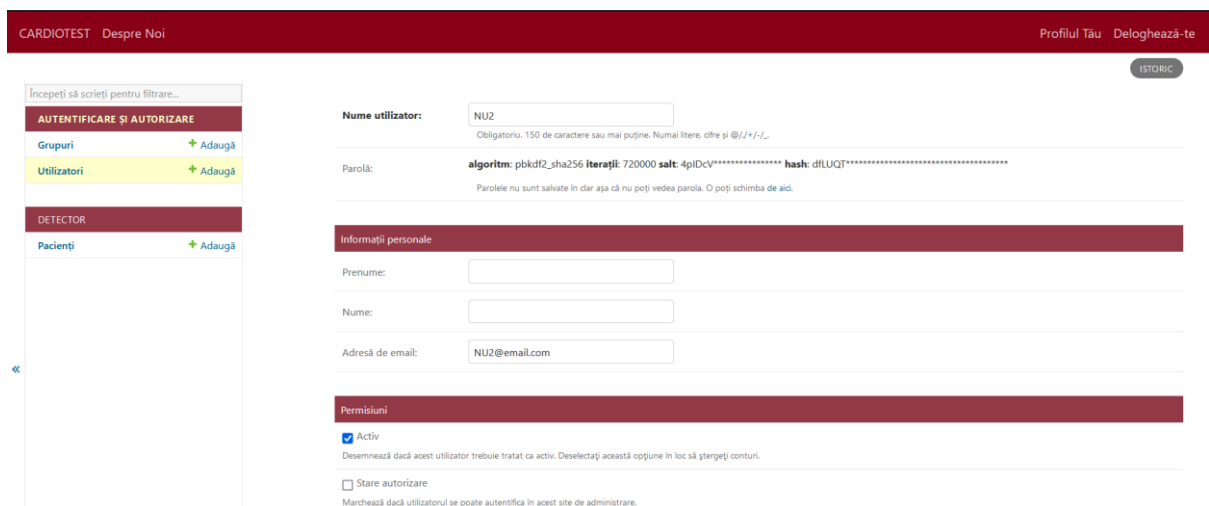


Figura 5.6

Pagina de administrare a grupurilor

Pagina de administrare a grupurilor afișează lista grupurilor create de administrator, care separă utilizatorii pe categorii cu permisiuni separate, trăsătură utilă în cazul în care grupurile de permisiuni predefinite, de utilizator normal, administrator sau staff, nu sunt suficiente. Aceste grupuri pot fi create, accesate și modificate prin adăugarea sau eliminarea de permisiuni din această interfață, după cum se poate observa în figura 5.7.

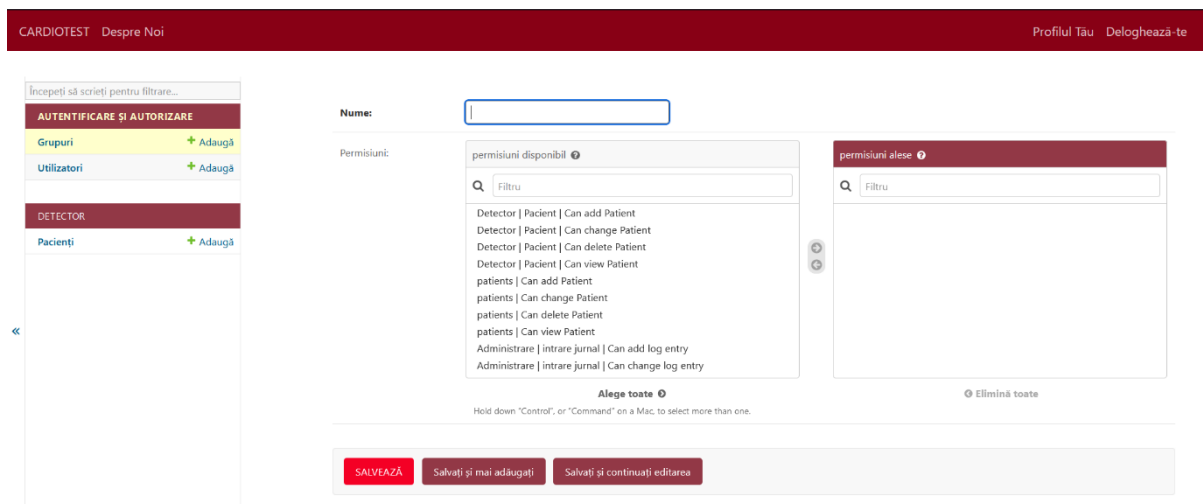


Figura 5.7

Pagina de administrare a pacienților

Pagina de administrare a pacienților permite vizualizarea listei de pacienți din baza de date, adăugarea intrărilor noi și ștergerea intrărilor existente, după cum se poate observa în figura 5.8. Administratorul poate de asemenea și modifica fiecare intrare a acestei liste, folosind formularul validat deschis când o intrare este accesată, cum se poate observa în figura 5.9.

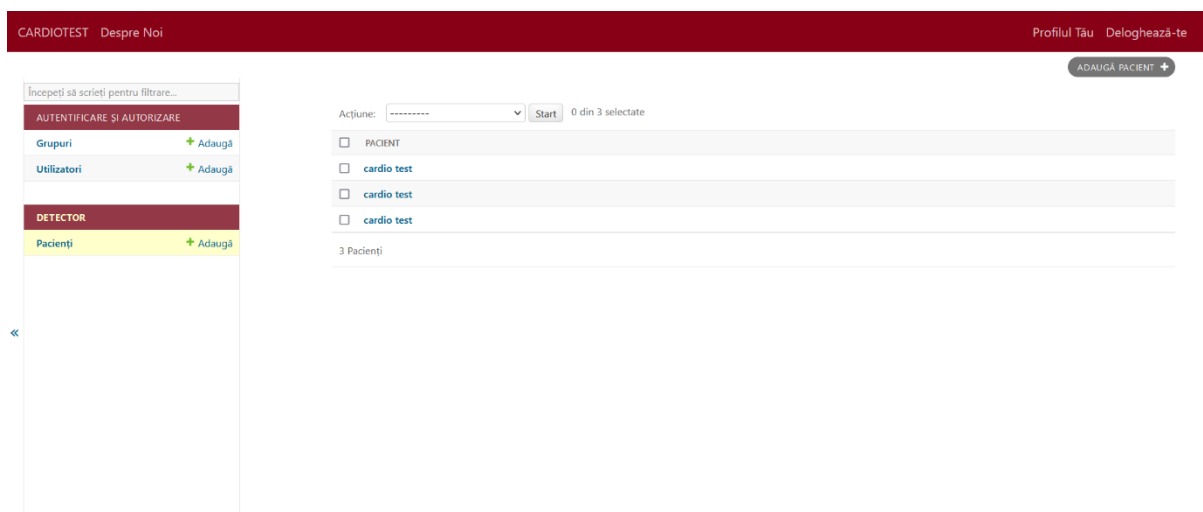


Figura 5.8

Figura 5.9

Modulul de autentificare

Pentru implementarea unui sistem de autentificare intuitiv, fluid și securizat am decis să utilizăm ca bază modulul oferit de Django, în principal datorită funcționalităților de gestionare a sesiunilor și de stocare securizată. Elementele standard utilizate din acest modul, pentru gestionarea logării, delogării și resetarea parolei prin email, au fost apoi modificate pentru a utiliza formulare consistente cu nevoile aplicației și sunt complementate de elementele create de noi pentru gestiunea salvării și modificării datelor necesare autentificării din interfața website-ului și a funcțiilor pentru notificările adiționale trimise prin email.

Deoarece aceste modele și formulare lucrează cu date similare și câmpurile definite de trebuie să fie definite similar, conținând, în funcție de datele cu care lucrează câmpuri din lista ['username', 'email', 'password1', 'password2'], pe care o putem observa în formularul UserRegisterForm folosit ca exemplu în figura 5.10.

```
class UserRegisterForm(UserCreationForm):
    email = forms.EmailField()

    class Meta:
        model = User
        fields = ['username', 'email', 'password1', 'password2']

    def __init__(self, *args, **kwargs):
        super(UserRegisterForm, self).__init__(*args, **kwargs)
        self.fields['username'].widget.attrs.update({'style': 'width: 100%'})
        self.fields['email'].widget.attrs.update({'style': 'width: 100%'})
        self.fields['password1'].widget.attrs.update({'style': 'width: 100%'})
        self.fields['password2'].widget.attrs.update({'style': 'width: 100%'})
```

Figura 5.10

Pagina de înregistrare

Pagina de înregistrare oferă un formular pentru utilizatorii noi unde aceștia își pot introduce informațiile personale și este structurată în maniera observabilă în figura 5.11.

CARDIOTEST Despre Noi Înregistrează-te Loghează-te

Înregistrează-te astăzi

Nume utilizator:

Obligatoriu. 150 de caractere sau mai puține. Numai litere, cifre și @/./+/-/_.

Email:

Parolă:

- Nu este recomandabil ca parola sa fie asemănătoare cu celelalte date personale.
- Parola trebuie să conțină cel puțin 8 caractere.
- Parola nu poate fi atât de comună.
- Parola nu poate fi doar cu numere.

Confirmare parolă:

Introduceți parola precedentă pentru confirmare.

Înregistrează-te

Ai deja un cont? [Loghează-te](#)

Figura 5.11

După completarea formularului view-ul primește datele utilizatorului, validează formularul pentru a asigura o complexitate minimă a parolei, unicitatea numelui de utilizator și validitatea formatelor datelor, iar dacă acestea sunt valide parola este hashed folosind algoritmul PBKDF2 care generează o valoare SHA256, și salvează datele utilizatorului, altfel returnează setul de erori din procesul de validare al formularului.

Implementarea view-ului poate fi observată în figura 5.12.

```
def register(request):
    if request.method == 'POST':
        form = UserRegisterForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, f'Your request has been submitted! Please wait for activation!')
            return redirect('login')
        else:
            form = UserRegisterForm()
    return render(request, 'Users/register.html', {'form': form})
```

Figura 5.12

Prin folosirea unui semnal, activat înainte ca datele utilizatorului să fie salvate contul utilizatorului este inițial marcat ca fiind inactiv, și un email este trimis administratorului pentru aprobarea accesului unui nou utilizator. Dacă contul este apoi aprobat de administrator utilizatorul este notificat prin email de acest fapt.

Metoda folosită pentru implementarea acestor elemente se regăsește în figura 5.13.

```
@receiver(pre_save, sender=User)
def request_admin_activation(sender, instance, **kwargs):
    if instance.pk is None:
        instance.is_active = False
        subject = "Account activation request"
        message = "User " + instance.username + " has requested account activation!"
        send_mail(subject, message, "cvddetector@noreply.com", ["superuser@email.com"], fail_silently=False,)

@receiver(pre_save, sender=User)
def send_activation_notification(sender, instance, **kwargs):
    if instance.is_active and User.objects.filter(pk=instance.pk, is_active=False).exists():
        subject = "Account activated"
        message = instance.username + " your account is now active!"
        send_mail(subject, message, "cvddetector@noreply.com", [instance.email], fail_silently=False,)
```

Figura 5.13

Pagina de autentificare

Pagina de autentificare permite utilizatorilor să se autentifice folosind email-ul și parola și este structurată similar cu pagina de înregistrare, după cum se poate observa și în figura 5.14.

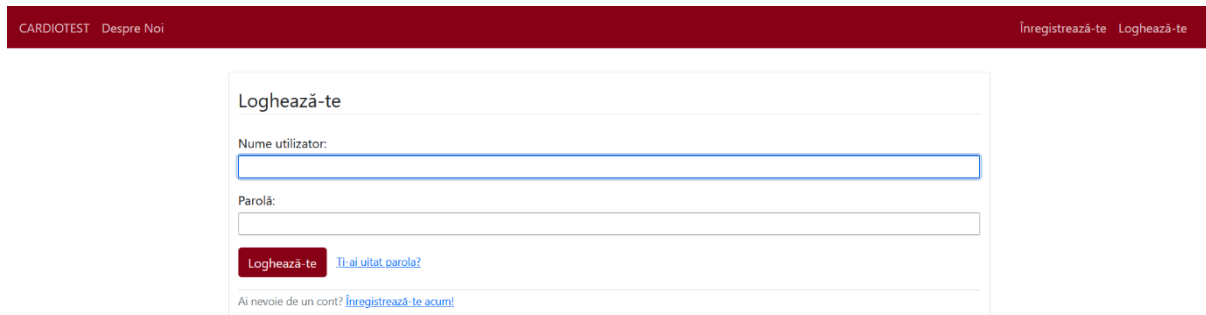


Figura 5.14

View-ul de autentificare, predefinit de Django, primește formularul cu datele de autentificare, pe care îl validează, iar dacă datele sunt corecte, crează un token de sesiune valabil pe o durată de timp sau până la deconectare și redirecționează utilizatorul către pagina principală, altfel dacă datele sunt incorecte returnează o eroare nespecifică pentru a nu oferi informații potențialilor atacatori.

Este important tot aici să menționăm că ,pentru securitate, Django gestionează protecția împotriva atacurilor CSRF și SQL Injection prin middleware și validări integrate.

Pagina de cerere resetare a parolei

Pagina de cerere resetare a parolei are o structură simplă și permite utilizatorilor să solicite resetarea parolei prin introducerea adresei de email, cum reiese și din figura 5.15.



The screenshot shows a web interface for a password reset. At the top, there is a dark red header bar with the text 'CARDIOTEST Despre Noi' on the left and 'Înregistrează-te Loghează-te' on the right. Below the header, the main content area is white. It features a form titled 'Resetează-ți Parola'. Inside the form, there is a label 'Email:' followed by a text input field. Below the input field is a red button with the text 'Resetează Parola'.

Figura 5.15

Componenta, predefinită în Django, trimite după completarea formularului un email către adresa specificată dacă se află printre utilizatori și este validă, cu un link marcat cu un token unic, care are un timp de expirare mult mai mic decât cel al token-urilor de sesiune și poate fi folosit de utilizator pentru a reseta parola.

Pagina de confirmare resetare a parolei

Pagina de confirmare resetare a parolei permite utilizatorilor, accesând link-ul primit pe email, să introducă o nouă parolă care este validată și salvată sub forma unui hash generat la creare, apoi invalidează token-ul folosit. Figura de mai jos redă formatul acestei pagini.



Resetează-ți Parola

Parolă nouă:

• Nu este recomandabil ca parola să fie asemănătoare cu celelalte date personale.
• Parola trebuie să conțină cel puțin 8 caractere.
• Parola nu poate fi atât de comună.
• Parola nu poate fi doar cu numere.

Confirmarea parolă nouă:

Resetează Parola

Figura 5.16

Pagina de administrare a profilului

Pagina de administrare a profilului permite utilizatorilor autentificați să își vizualizeze și actualizeze numele de utilizator sau email-ul asociat contului, prin intermediul unui formular simplu, vizibil în figura de mai jos. După ce utilizatorul face modificările și apasă butonul de salvare, datele sunt validate și actualizate în baza de date.



Profilul Tău

Nume utilizator:

NU2

Obligatoriu. 150 de caractere sau mai puține. Numai litere, cifre și @/./+/-/_.

Email:

NU2@email.com

Actualizează

Figura 5.17

Pagina de deautentificare

Pagina de deautentificare, vizibilă în figura 5.18, este pagina către care utilizatorul este redirecționat când se deconectează prin apăsarea butonului asociat acestei acțiuni, care trimite un formular de ștergere a token-ului asociat sesiunii active din baza de date.

Ai fost deconectat!

[Loghează-te din nou!](#)

Figura 5.18

Modulul de Pacienți

Pagina principală

Pagina principală are ca și componente principale, după cum putem observa în figura 5.19, un tabel paginat care afișează datele pacienților și adaugă un buton de ștergere fiecărei intrări, o bară laterală cu un set de filtre cu ajutorul căruia se pot căuta pacienți cu parametri specifici și un formular de adăugare a datelor pacientului, lucruri care permit utilizatorului să gestioneze datele pacienților cu ușurință.

NSS	Prenom	Nume	Email	Vârsta	Sex	Înălțime	Greutate	TAS	TAD	Colesterol	Glucoză	Fumător	Băutor	Activitate	Risc Boală	Data Adăugare	Acțiuni
123456789	cardio	test	cardio.test@email.com	17000	1	150,0	200,0	130	140	1	1	1	0	0,8870490193367004	22/06/2024, 04:45	Stergere	
123456789	cardio	test	cardio.test@email.co	39000	1	1,0	1,0	1	1	2	2	0	0	0	0,744855463504791	22/06/2024, 04:35	Stergere
123456789	cardio	test	cardio.test@email.com	20000	1	190,0	130,0	110	130	3	3	0	0	1	0,89999999761581421	19/06/2024, 07:17	Stergere

Figura 5.19

Tabela utilizată pentru stocarea datelor pacienților conține două categorii de date, folosite pentru stocarea datelor de contact și a identificatorilor pacienților, și anume NSS, Prenom, Nume și Email, respectiv datele medicale asociate pacientului, Vârsta, Sex, Înălțime, Greutate, TAS, TAD, Colesterol, Glucoză, Fumător, Băutor, Activitate, folosite pentru generarea câmpului Risc_Boală care apelează modelul utilizat pentru predicție și salvează rezultatul returnat de acesta. Conversia datelor din obiecte în intrări ale bazei de date și viceversa pentru procesare este realizată cu ajutorul modelului Pacient, vizibil în figura 5.20.

Adăugarea și ștergerea pacienților în baza de date se face prin intermediul elementelor dedicate din template, care trimit request-uri ce conțin natura acțiunii ce trebuie executate de view. View-ul mai apoi procesează și identifică natura acestor cerințe, alături de datele de context pentru a șterge intrarea dorită folosind identificatorul acesteia, sau a salva formularul cu datele pacienților și riscul care reiese din acestea prin apelarea modelului. Implementarea acestei componente poate fi observată în figura 5.21, iar a formularului folosit la adăugare în figura 5.22.

Pentru componenta de filtrare a listei de pacienți am decis, datorită flexibilității metodei și deoarece salvarea datelor nu este necesară, să referențiem câmpurile care conțin parametrii de filtrare doriți de utilizator folosind identificatorii lor direcți, fără a implementa un formular Django, după cum se poate observa în figura 5.23.

Pentru definirea tabelului paginat utilizăm un obiect de tip Paginator, vizibil în figura 5.24, care preia setul de date cu pacienții doriți și numărul de elemente dorit pe pagină, și îl divizează conform cerințelor pentru a putea fi parcurs și afișat într-o manieră plăcută vizual.

```
class Pacient(models.Model):
    NSS = models.CharField(max_length=18)
    Prenume = models.CharField(max_length=100)
    Nume = models.CharField(max_length=100)
    Email = models.EmailField()
    Vârsta = models.IntegerField()
    Sex = models.IntegerField(choices=[(1, 'Bărbat'), (2, 'Femeie')])
    Înălțime = models.FloatField()
    Greutate = models.FloatField()
    TAS = models.IntegerField()
    TAD = models.IntegerField()
    Colesterol = models.IntegerField(choices=[(1, 'Normal'), (2, 'Above Normal'), (3, 'Well Above Normal')])
    Glucoză = models.IntegerField(choices=[(1, 'Normal'), (2, 'Above Normal'), (3, 'Well Above Normal')])
    Fumător = models.IntegerField(choices=[(0, 'Nu'), (1, 'Da')])
    Băutor = models.IntegerField(choices=[(0, 'Nu'), (1, 'Da')])
    Activitate = models.IntegerField(choices=[(0, 'Nu'), (1, 'Da')])
    Risc_Boală = models.FloatField(blank=True, default=None)
    Data_Adăugare = models.DateTimeField(auto_now_add=True)

    class Meta:
        verbose_name = "Pacient"
        verbose_name_plural = "Pacienți"

    def __str__(self):
        return f'{self.Prenume} {self.Nume}'
```

Figura 5.20

```

model = joblib.load('Models/Models/XG_Boost/xgb_model.pkl')
scaler = joblib.load('Models/Models/scaler.pkl')
model_columns = ['Vârsta', 'Sex', 'Înălțime', 'Greutate', 'TAS', 'TAD', 'Colesterol', 'Glucoză', 'Fumător', 'Băutor', 'Activitate']
continuous_columns = ['Vârsta', 'Înălțime', 'Greutate', 'TAS', 'TAD']

if request.method == 'POST' and 'action' in request.POST:
    action = request.POST.get('action')
    if action == 'delete':
        patient_id = request.POST.get('patientId')
        patient = get_object_or_404(Pacient, id=patient_id)
        patient.delete()
        return redirect('home')
    elif action == 'add_patient':
        form = FormularPacient(request.POST)
        if form.is_valid():
            new_patient = form.save(commit=False)
            patient_data = {col: getattr(new_patient, col, 0) for col in model_columns}
            patient_df = pd.DataFrame([patient_data])
            continuous_data = patient_df[continuous_columns]
            categorical_data = patient_df.drop(columns=continuous_columns)
            normalized_continuous_data = scaler.transform(continuous_data)
            normalized_patient_data = pd.DataFrame(normalized_continuous_data, columns=continuous_columns).join(categorical_data.reset_index(drop=True))
            model_input = normalized_patient_data.iloc[0].tolist()
            new_patient.Risc_Boală = model.predict([model_input])[0]
            new_patient.save()
            return redirect('home')
form = FormularPacient()

```

Figura 5.21

```

class FormularPacient(forms.ModelForm):
    class Meta:
        model = Pacient
        fields = '__all__'
        widgets = {
            'Sex': forms.Select(
                choices=((1, 'Bărbat'), (2, 'Femeie')),
                attrs={'style': 'width: 100%;'}
            ),
            'Colesterol': forms.Select(
                choices=((1, 'Normal'), (2, 'Above Normal'), (3, 'Well Above Normal')),
                attrs={'style': 'width: 100%;'}
            ),
            'Glucoză': forms.Select(
                choices=((1, 'Normal'), (2, 'Above Normal'), (3, 'Well Above Normal')),
                attrs={'style': 'width: 100%;'}
            ),
            'Fumător': forms.Select(
                choices=((0, 'Nu'), (1, 'Da')),
                attrs={'style': 'width: 100%;'}
            ),
            'Băutor': forms.Select(
                choices=((0, 'Nu'), (1, 'Da')),
                attrs={'style': 'width: 100%;'}
            ),
            'Activitate': forms.Select(
                choices=((0, 'Nu'), (1, 'Da')),
                attrs={'style': 'width: 100%;'}
            ),
            'NRS': forms.TextInput(attrs={'style': 'width: 100%;'}),
            'Prenume': forms.TextInput(attrs={'style': 'width: 100%;'}),
            'Nume': forms.TextInput(attrs={'style': 'width: 100%;'}),
            'Email': forms.EmailInput(attrs={'style': 'width: 100%;'}),
            'Vârsta': forms.NumberInput(attrs={'style': 'width: 100%;'}),
            'Înălțime': forms.NumberInput(attrs={'style': 'width: 100%;'}),
            'Greutate': forms.NumberInput(attrs={'style': 'width: 100%;'}),
            'TAS': forms.NumberInput(attrs={'style': 'width: 100%;'}),
            'TAD': forms.NumberInput(attrs={'style': 'width: 100%;'}),
            'Risc_Boală': forms.NumberInput(attrs={'style': 'width: 100%;'}),
            'Data_Adăugare': forms.DateInput(attrs={'style': 'width: 100%;'}),

```

Figura 5.22

```

qs = Patient.objects.all()

filters = {
    'NMS__icontains': request.GET.get('NMS_contains'),
    'Prenume__icontains': request.GET.get('Prenume_contains'),
    'Nume__icontains': request.GET.get('Nume_contains'),
    'Email__icontains': request.GET.get('Email_contains'),
    'Înălțime__gte': request.GET.get('Înălțime_gte'),
    'Înălțime__lte': request.GET.get('Înălțime_lte'),
    'Greutate__gte': request.GET.get('Greutate_gte'),
    'Greutate__lte': request.GET.get('Greutate_lte'),
    'TMS__gte': request.GET.get('TMS_gte'),
    'TMS__lte': request.GET.get('TMS_lte'),
    'TMD__gte': request.GET.get('TMD_gte'),
    'TMD__lte': request.GET.get('TMD_lte'),
    'Cholesterol': request.GET.get('Cholesterol'),
    'Glucoză': request.GET.get('Glucoză'),
    'Fumător': request.GET.get('Fumător'),
    'Băutor': request.GET.get('Băutor'),
    'Activ': request.GET.get('Activ'),
    'Risc_Boală__gte': request.GET.get('Risc_Boală_gte'),
    'Risc_Boală__lte': request.GET.get('Risc_Boală_lte'),
    'Sex': request.GET.get('Sex'),
    'Vârsta__gte': request.GET.get('Vârsta_gte'),
    'Vârsta__lte': request.GET.get('Vârsta_lte'),
    'Dată_Adăugare_date__gte': request.GET.get('Dată_Adăugare_date_gte'),
    'Dată_Adăugare_date__lte': request.GET.get('Dată_Adăugare_date_lte'),
}

for filter_key, filter_value in filters.items():
    if filter_value:
        qs = qs.filter(**(filter_key: filter_value))

```

Figura 5.23

```

qs = qs.order_by('-Dată_Adăugare')

paginator = Paginator(qs, 25)
page = request.GET.get('page')
try:
    patients = paginator.page(page)
except PageNotAnInteger:
    patients = paginator.page(1)
except EmptyPage:
    patients = paginator.page(paginator.num_pages)

```

Figura 5.24

Pagina detalii pacient

Pagina detalii pacient conține trei elemente principale, separabile vizual. Prima componentă constă dintr-un tabel care conține o listă a intrărilor anterioare ale pacientului și predicțiile reieșite din acestea. Cea de-a doua componentă constă dintr-o subsecțiune a paginii care conține două tab-uri, prima dintre care poate fi observată în figura 5.25 și reprezintă un grafic care compară rezultatele istorice ale pacientului, în funcție de procentajul de pacienți peste care datele din câmpurile cu caracter medical îl plasează, separate pe categorii de la Normal până la Mult Peste Limita Normală, sau binare, Da sau Nu în funcție de prezența unor trăsături. Cel de al doilea tab conține un tabel care, pentru fiecare dată calculează și organizează descrescător contribuția caracteristicilor pacientului la riscul de boală, calculată folosind valorile câmpurilor cu caracter medical și greutatea lor în modelul de predicție, după cum se poate observa în figura 5.26. Cea de a treia componentă separabilă constă într-un formular de filtrare care poate fi folosit pentru a selecta coloanele vizibile în graficul generat pentru utilizator.

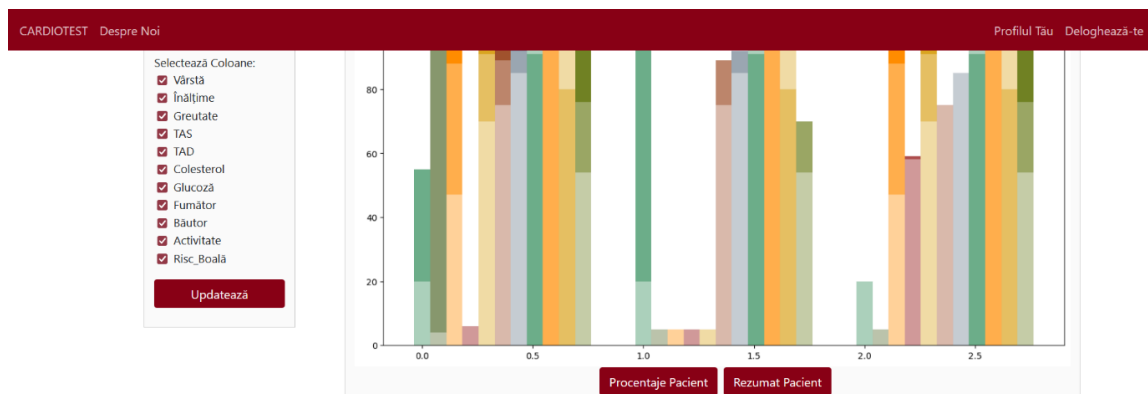


Figura 5.25

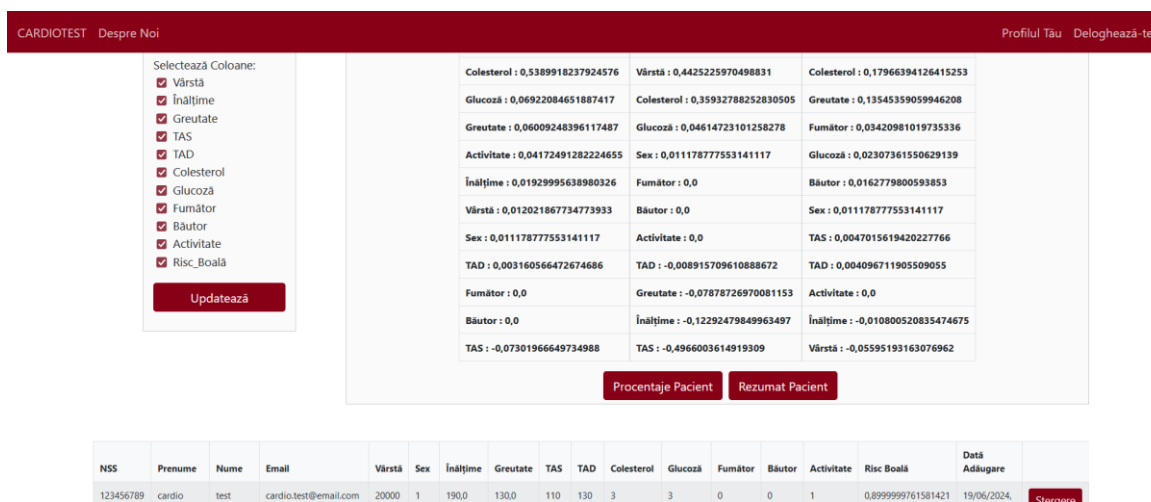


Figura 5.26

Componenta tabel, ce conține datele istorice ale pacientului se folosește de identificatorul unic al pacientului, parte din link-ul folosit pentru accesarea paginii asociate fiecărui pacient și parametru al view-ului din backend, pentru a obține datele acestuia, a le ordona descrescător și a le afișa într-o manieră similară cu pagina principală, după cum reiese și din figurile de mai sus.

Pentru generarea graficului dorit este necesar ca în prealabil să calculăm procentajul de pacienți cu valori sub cele ale pacientului, categorisite în maniera descrisă anterior. Acest lucru este realizat în cadrul funcției calculate_percentiles, vizibilă în figura 5.27, care returnează un dicționar ce conține, în funcție de tipul de date al coloanelor cu caracter medical, valori calculate astfel: pentru valorile binare returnează procentajul de utilizatori din fiecare categorie, pentru valorile ce reprezintă categorii returnează procentajele de utilizatori din categoriile aflate sub sau egale cu categoria pacientului pentru a fi vizibilă distribuția utilizatorilor pe aceste categorii, iar pentru valorile continue returnează procentajul pacienți cu valori aflate sub cele ale pacientului evaluat, separând categoriile folosind breakpoint-uri de valoare pentru ceea

ce poate fi considerat valoare Normală, valoare Peste Limita Normală, și valoare Mult Peste Limita Normală.

```
def calculate_percentiles(patient_values, training_data, selected_columns):
    detailed_percentages = []
    bottom_limits = {
        'Vârsta': [0, 17000, 23000],
        'Înălțime': [0, 150, 180],
        'Greutate': [0, 70, 90],
        'TAS': [0, 120, 140],
        'TAD': [0, 80, 90],
        'Risc_Boală': [0, 0.5, 0.8],
    }
    for values in patient_values:
        patient_percentages = {}
        for field in selected_columns:
            patient_value = values[field]
            field_percentages = {}
            if field in ['Functor', 'BAutor', 'Activitate']:
                total_count = len(training_data)
                categories = {'Da': 1, 'Nu': 0}
                field_percentages = {category: round((training_data[field] == value).sum() / total_count * 100)
                                     for category, value in categories.items()}
                field_percentages = {k: v for k, v in field_percentages.items()}
            elif field in ['Glucoză', 'Cholesterol']:
                categories = {1: 'Normal', 2: 'Peste Limita Normală', 3: 'Mult Peste Limita Normală'}
                field_percentages = {desc: round((training_data[field] == value).sum() / len(training_data) * 100)
                                     for value, desc in categories.items()}
                patient_category = categories[patient_value]
                field_percentages = {k: v for k, v in field_percentages.items() if list(categories.values()).index(k) <= list(categories.values()).index(patient_category)}
            else:
                total_count = len(training_data)
                limits = bottom_limits.get(field, [0])
                categories = ['Normal', 'Peste Limita Normală', 'Mult Peste Limita Normală']
                cumulative_percentage = 0
                cumulative_field_percentages = []
                for i in range(len(limits) - 1):
                    category_percentage = round(((training_data[field] > limits[i]) & (training_data[field] <= limits[i+1])).sum() / total_count * 100)
                    cumulative_percentage += category_percentage
                    cumulative_field_percentages.append(cumulative_percentage)
                cumulative_field_percentages.append(cumulative_percentage + round((training_data[field] > limits[-1]).sum() / total_count * 100))
                below_percentage = round((training_data[field] < patient_value).sum() / total_count * 100)
                previous_sum = 0
                for i, category in enumerate(categories):
                    top_end = cumulative_field_percentages[i]
                    field_percentages[category] = min(below_percentage, top_end) - previous_sum
                    previous_sum += field_percentages[category]
                if field_percentages['Normal'] < 5 and field_percentages['Peste Limita Normală'] == 0 and field_percentages['Mult Peste Limita Normală'] == 0:
                    field_percentages['Normal'] = 5
                patient_percentages[field] = (patient_value, field_percentages)
            detailed_percentages.append(patient_percentages)
    return detailed_percentages
```

Figura 5.27

Generarea propriu-zisă a graficului se realizează în cadrul funcției `generate_column_chart`, din figura 5.28, care parcurge datele generate de funcția `calculate_percentiles` și le folosește pentru a genera graficul dorit. Pentru generarea acestui grafic definim culorile fiecărei coloane și nuanțele luate de fiecare categorie, care sunt parcurse, dacă sunt selectate în componenta de filtrare, în ordine, organizate în funcție modul în care sunt corelate cu riscul și afișate în grafic.

Pentru implementarea selectorului de coloane vizibile în grafic am utilizat o listă simplă de coloane a căror valori pot fi modificate din interfață.

Logica componentei din tab-ul folosit în explicarea riscului este implementată în cadrul funcției `generate_risk_reasons`, vizibilă în figura 5.29, care încarcă modelul folosit la predicție preluând greutatea caracteristicilor folosite de acesta și înmulțindu-le cu valorile normalizate ale datelor pacientului, folosite la antrenarea modelului, pentru a determina contribuția la risc a fiecărei caracteristici și a le returna pentru a fi afișate utilizatorului.

```

def generate_column_chart(patient_data, percentages, selected_columns):
    dates = [pd.to_datetime(data.Dată_Adăugare) for data in patient_data]
    unique_dates = sorted(set(dates))
    fig, ax = plt.subplots(figsize=(14, 7))
    bar_width = 0.8 / len(selected_columns)
    columns = {
        "Vârsta": {"color": "#2E8B57", "opacity": {"Normal": 0.4, "Peste Limita Normală": 0.7, "Mult Peste Limita Normală": 1.0}},
        "Înălțime": {"color": "#556B2F", "opacity": {"Normal": 0.4, "Peste Limita Normală": 0.7, "Mult Peste Limita Normală": 1.0}},
        "Greutate": {"color": "#FF8C00", "opacity": {"Normal": 0.4, "Peste Limita Normală": 0.7, "Mult Peste Limita Normală": 1.0}},
        "TAS": {"color": "#8B0000", "opacity": {"Normal": 0.4, "Peste Limita Normală": 0.7, "Mult Peste Limita Normală": 1.0}},
        "TAD": {"color": "#DAA520", "opacity": {"Normal": 0.4, "Peste Limita Normală": 0.7, "Mult Peste Limita Normală": 1.0}},
        "Colesterol": {"color": "#A0522D", "opacity": {"Normal": 0.4, "Peste Limita Normală": 0.7, "Mult Peste Limita Normală": 1.0}},
        "Glucoză": {"color": "#708080", "opacity": {"Normal": 0.4, "Peste Limita Normală": 0.7, "Mult Peste Limita Normală": 1.0}},
        "Risc_Boală": {"color": "#708080", "opacity": {"Normal": 0.4, "Peste Limita Normală": 0.7, "Mult Peste Limita Normală": 1.0}},
        "Fumător": {"color": "#2E8B57", "opacity": {"Nu": 0.7, "Da": 0.4}},
        "Băutor": {"color": "#FF8C00", "opacity": {"Nu": 0.7, "Da": 0.4}},
        "Activitate": {"color": "#DAA520", "opacity": {"Nu": 0.7, "Da": 0.4}},
    }

    for i, field in enumerate(selected_columns):
        if field not in columns:
            continue
        category_opacities = columns[field]["opacity"]
        field_values = {category: [0] * len(unique_dates) for category in category_opacities.keys()}
        for j in range(len(patient_data)):
            patient_values, field_percentages = percentages[j].get(field, (0, {}))
            for cat, percent in field_percentages.items():
                if cat in field_values:
                    field_values[cat][dates.index(patient_data[j].Dată_Adăugare)] += percent
        base_color = columns[field]["color"]
        ordered_categories = {
            "Vârsta": ["Normal", "Peste Limita Normală", "Mult Peste Limita Normală"],
            "Înălțime": ["Normal", "Peste Limita Normală", "Mult Peste Limita Normală"],
            "Greutate": ["Normal", "Peste Limita Normală", "Mult Peste Limita Normală"],
            "TAS": ["Normal", "Peste Limita Normală", "Mult Peste Limita Normală"],
            "TAD": ["Normal", "Peste Limita Normală", "Mult Peste Limita Normală"],
            "Colesterol": ["Normal", "Peste Limita Normală", "Mult Peste Limita Normală"],
            "Glucoză": ["Normal", "Peste Limita Normală", "Mult Peste Limita Normală"],
            "Risc_Boală": ["Normal", "Peste Limita Normală", "Mult Peste Limita Normală"],
            "Fumător": ["Nu", "Da"],
            "Băutor": ["Nu", "Da"],
            "Activitate": ["Nu", "Da"],
        }
        cumulative_values = [0] * len(unique_dates)
        for category in ordered_categories[field]:
            if category in field_values:
                values = field_values[category]
                opacity = category_opacities[category]
                color = plt.cm.colors.to_rgba(base_color, alpha=opacity)
                ax.bar(
                    np.arange(len(unique_dates)) + i * bar_width,
                    values,
                    bar_width,
                    bottom=cumulative_values,
                    label=f'{field} {category}',
                    color=color
                )
                cumulative_values = [sum(x) for x in zip(cumulative_values, values)]

    buffer = io.BytesIO()
    plt.savefig(buffer, format='png', bbox_inches='tight')
    buffer.seek(0)
    plt.close(fig)
    graph_base64 = base64.b64encode(buffer.getvalue()).decode('utf-8')
    return graph_base64

```

Figura 5.28

```

model = joblib.load('Models/Models/XG_Boost/xgb_model.pkl')
scaler = joblib.load('Models/Models/scaler.pkl')
baseline = pd.read_csv('Models/Datasets/xgb_enhanced_cardio_train.csv')

def generate_risk_reasons(patient):
    model_features = model.feature_names_in_
    feature_importances = model.feature_importances_
    continuous_columns = ['Vârsta', 'Înălțime', 'Greutate', 'TAS', 'TAD']
    categorical_columns = ['Sex', 'Colesterol', 'Glucoză', 'Fumător', 'Băutor', 'Activitate']
    patient_data = {
        'Vârsta': patient.Vârsta,
        'Sex': patient.Sex,
        'Înălțime': patient.Înălțime,
        'Greutate': patient.Greutate,
        'TAS': patient.TAS,
        'TAD': patient.TAD,
        'Colesterol': patient.Colesterol,
        'Glucoză': patient.Glucoză,
        'Fumător': patient.Fumător,
        'Băutor': patient.Băutor,
        'Activitate': patient.Activitate
    }
    patient_df = pd.DataFrame([patient_data])
    continuous_data = patient_df[continuous_columns]
    categorical_data = patient_df[categorical_columns]
    normalized_continuous_data = scaler.transform(continuous_data)
    normalized_patient_data = pd.DataFrame(normalized_continuous_data, columns=continuous_columns).join(categorical_data.reset_index(drop=True))
    normalized_patient_data = normalized_patient_data[model_features]
    model_input = normalized_patient_data.iloc[0].tolist()
    reasons = [col: feature_importances[i] * model_input[i] for i, col in enumerate(model_features)]
    sorted_reasons = sorted(reasons.items(), key=lambda item: item[1], reverse=True)
    return sorted_reasons

```

Figura 5.29

Pagina “Despre noi”

Pagina “Despre noi”, din figura 5.30, este menită să informeze utilizatorii cu privire la natura aplicației și contextul de utilizare în care această este intenționată a fi folosită, în particular că acesta trebuie folosită nu ca înlocuitor ci alături de sfatul specialiștilor medicali.

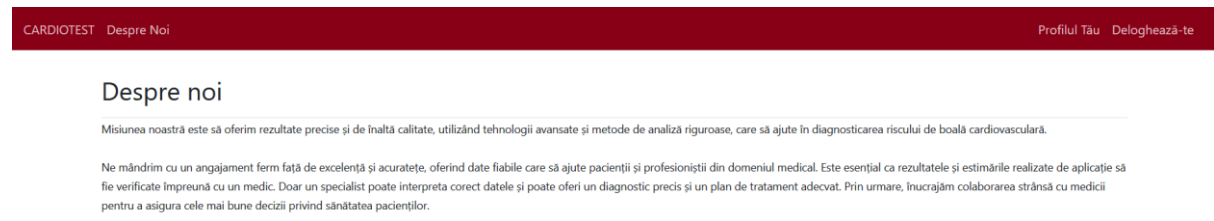


Figura 5.30

Capitolul VI. Concluzii

În cadrul proiectului de diplomă am implementat o aplicație de gestionare a datelor pacienților unui centru de analize cu funcționalități de detectare a bolilor cardiovasculare și generare de grafice.

Aplicația dezvoltată utilizează numeroase funcționalități oferite de Django, inclusiv moștenirea și integrarea cu clasele standard ale framework-ului, lucru care a permis dezvoltarea facilă, simplă unei aplicații web complexe și bine securizate.

În ceea ce privește utilizatorii, aplicația se asigură că utilizatorii pot gestiona propriile conturi prin implementarea funcționalităților de resetare a parolei, schimbare a email-ului și de a aplica pentru un cont nou.

Securitatea aplicației este asigurată de sistemul de autentificare care folosește token-uri unice și de sesiune, cu durate maxime, generate și verificate automat. Aceste token-uri sunt criptate și validate de procese de tip middleware de Django pentru a preveni accesul la componente la care utilizatorul nu are acces. Sistemul de resetarea parolei este de asemenea implementat folosind token-uri, dar acestea sunt eliminate după accesarea link-ului și au o durată maximă mai scurtă pentru a preveni accesul nedorit.

Administratorii aplicației pot utiliza de un panou de control complex, vizibil doar utilizatorilor desemnați de aceștia pentru a aproba conturile noi și gestiona utilizatorii aplicației. Interfața de administrare nativă este modificată, pentru a elimina elementele nedorite și are un aspect unitar cu restul aplicației datorită implementării clasei CustomAdmin care moștenește de la acesta.

În ceea ce privește componenta de stocare și analiză a datelor pacienților aplicația salvează datele folosind formulare validate pentru a evita introducerea datelor greșite. Graficele și vizualizările generate pe baza datelor medicale sunt generate dinamic, conținând controale granulare pentru coloanele vizibile, iar analiza datelor medicale a fost realizată cu ajutorul algoritmilor de inteligență artificială și oferă explicații pentru rezultatul generat folosindu-se de greutățile returnate de algoritm și valoarea normalizată a datelor pacientului.

Pentru antrenarea componentei de inteligență artificială am folosit un algoritm de Isolation Forest pentru detectarea anomaliilor și un algoritm de normalizare standard pentru a asigura că algoritmul tratează toate coloanele corect indiferent de intervalul pe care se află. Ulterior, aceste date au fost transformate din date de clasificare în date de regresie folosind algoritmi de prezicere a probabilității compatibili cu algoritmi de regresie utilizați în predicție și procesate folosind modele de Linear Regression, Random Forest și XGBoost, ale căror performanțe și acuratețe au fost evaluate pentru a asigura generarea unor predicții utile. În final, XGBoost a fost algoritmul ales pentru analiza principală datorită preciziei sale superioare, fără pierderi de performanță odată ce modelul este salvat.

În viitor funcționalitățile aplicației pot fi extinse prin integrarea unor metode avansate de analiză a datelor medicale care să folosească rețele neuronale și algoritmi de deep learning, augmentate de un algoritm de explicare a modelelor cu valori Shapley. Pentru îmbunătățirea securității aplicația poate fi extinsă cu algoritmi de anonimizarea a datelor pacientului, backup-uri frecvente ale bazei de date și augmentarea sistemelor de logging. De asemenea, se pot înregistra noi categorii de utilizatori, cum ar fi specialiști medicali și analiști de date, în cazul în care acest lucru este necesar.

Bibliografie

- [1]. HTML5, <https://developer.mozilla.org/en-US/docs/Web/HTML>, (19.05.2024)
- [2]. CSS3, <https://developer.mozilla.org/en-US/docs/Web/CSS>, (19.05.2024)
- [3]. Javascript, <https://developer.mozilla.org/en-US/docs/Learn>, (19.05.2024)
Getting_started_with_the_web/JavaScript_basics, (19.05.2024)
- [4]. Bootstrap, <https://getbootstrap.com/docs/5.0/getting-started/introduction>, (19.05.2024)
- [5]. Python, <https://docs.python.org/3/tutorial/index.html>, (23.05.2024)
- [6]. pandas, <https://pandas.pydata.org/docs/>, (23.05.2024)
- [7]. numpy, <https://numpy.org/doc/>, (23.05.2024)
- [8]. Joblib, <https://joblib.readthedocs.io/en/stable>, (23.05.2024)
- [9]. ScikitLearn, <https://scikit-learn.org/0.21/documentation.html>, (23.05.2024)
- [10]. XGBoost (Introducere), <https://xgboost.readthedocs.io/en/stable/python/index.html>,
(23.05.2024)
- [11]. Matplotlib (Introducere),
https://matplotlib.org/stable/users/explain/quick_start.html, (23.05.2024)
- [12]. Django, <https://docs.djangoproject.com/en/5.0>, (25.05.2024)
- [13]. SQLite, <https://www.sqlite.org/docs.html>, (26.05.2024)
- [14]. IsolationForest (Considerații),
<https://medium.com/codex/isolation-forest-outlier-detection-simplified-5d938548bb5c>,
(30.05.2024)
- [15]. IsolationForest (Implementare),
<https://scikitlearn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>,
(30.05.2024)
- [16]. StandardScaler (Considerații),
<https://medium.com/@hhuseyincosgun/which-data-scaling-technique-should-i-use-a1615292061est>, (31.05.2024)
- [17]. StandardScaler (Implementare),
<https://scikitlearn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>,
(31.05.2024)
- [18]. Metrice de Evaluaare,
<https://www.geeksforgeeks.org/metrics-for-machine-learning-model>, (01.06.2024)
- [19]. Cross Validation, <https://www.geeksforgeeks.org/cross-validation-machine-learning>,
(04.06.2024)
- [20]. Linear Regression (Considerații), <https://www.ibm.com/topics/linear-regression>,
(05.06.2024)
- [21]. Logistic Regression (Considerații),
<https://developers.google.com/machinelearningcrash-course/logistic-regression/calculating-a-probability>, (06.06.2024)

- [22]. Linear Regression (Implementare),
https://scikitlearn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html, (05.06.2024)
- [23]. Logistic Regression (Implementare),
https://scikitlearn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html, (06.06.2024)
- [24]. Random Forest (Considerații), <https://www.ibm.com/topics/random-forest>, (08.06.2024)
- [25]. Random Forest Classifier (Implementare),
<https://scikitlearn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>, (08.06.2024)
- [26]. Random Forest Regressor (Implementare),
<https://scikitlearn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>, (11.06.2024)
- [27]. XGBoost (Considerații), <https://www.ibm.com/topics/xgboost>, (11.06.2024)
- [28]. XGBoost (Implementare)
https://xgboost.readthedocs.io/en/stable/python/python_intro.html#prediction, (11.06.2024)
- [29]. Arhitectura MVT, <https://www.geeksforgeeks.org/django-project-mvt-structure>, (14.06.2024)